

GeoFare – 3. Defensive Programming

Software Engineering Activity: Identification of code that implements and does not fully implement Defensive Programming.

1. Meaning of Defensive Programming

Defensive programming anticipates invalid input, unexpected states, missing data, and operation failures so the application can respond safely instead of crashing or behaving unpredictably.

2. Code That Implements Defensive Programming

File / Code	Finding	Assessment
LoginScreen.kt	Checks whether email or password is empty before attempting authentication.	Implements
LoginScreen.kt	Uses Firebase authentication completion handling and displays an error when authentication fails.	Implements
LoginScreen.kt	Handles specific Firebase exceptions such as FirebaseAuthInvalidUserException and FirebaseAuthInvalidCredentialsException .	Implements
RegisterScreen.kt	Validates required fields, email format, minimum password length, and password confirmation before registration.	Implements
RegisterScreen.kt	Checks auth.currentUser?.uid before saving the user document.	Implements
RegisterScreen.kt	Uses addOnFailureListener when saving user data to Firestore.	Implements
HomeScreen.kt	Uses auth.currentUser?.uid ?: return@LaunchedEffect , safely stopping the Firestore request when no authenticated user exists.	Implements
SplashScreen.kt	Checks whether currentUser exists before deciding whether to navigate to Home or Login.	Implements

3. Code That Does Not Fully Implement Defensive Programming

File / Code	Finding	Assessment
HomeScreen.kt	The Firestore user lookup has an addOnSuccessListener but no addOnFailureListener . A failed database request is therefore not explicitly handled.	Needs improvement
HomeScreen.kt	The Emergency button has no actual safety action or error-handling behavior.	Needs improvement
LoginScreen.kt	The code checks empty fields but does not trim whitespace from the email/password input before processing.	Needs improvement
RegisterScreen.kt	The code validates the email format but does not trim the email before creating the Firebase account.	Needs improvement
FareCalculationScreen.kt	The actual fare calculation has not been implemented, so invalid distance/fare input cannot yet be handled.	Incomplete
TripMonitoringScreen.kt	Actual GPS/route monitoring is not implemented, so location errors and unavailable location data are not handled.	Incomplete
TripHistoryScreen.kt	No database retrieval is implemented, so there is currently no handling for missing, empty, or failed trip-history data.	Incomplete
ReportConcernScreen.kt	No report submission is implemented, so input validation and submission failure handling are not yet present.	Incomplete

4. Overall Finding

GeoFare already demonstrates defensive programming in authentication, registration validation, null checks, and Firestore failure handling. The main improvements needed are additional failure listeners, input normalization, and defensive handling for the unfinished features.